

CS141 Coursework 2 Report

u5685831

Overview

This project implements λ GPT, a REPL-based chatbot which parses natural language input into structured requests and produces appropriate responses.

Beyond the base specification, I focused on extending the system with permanent memory and a wide range of stock market features using external API calls.

Architecture & Module Design

As the project grew, I split the code into multiple modules to maintain clarity and separation of concerns. The main modules are:

- **TUI.hs** – runs the REPL and initialises memory
- **Parser.hs** – parses user input into structured requests
- **Request.hs** – defines all shared data types
- **Responder.hs** – handles logic and produces responses
- **Memory.hs** / **API.hs** – extended functionality (discussed later)

Separating **Request** into its own module avoids circular dependencies between parsing and responding. Making the project easier to expand, new features only required new request types and corresponding handlers.

Core Functionality

The initial exercises (greetings and date handling) are relatively simple and mainly involve direct parsing and response mapping, so they are not discussed in detail.

Expression Evaluation

Expressions are parsed in natural language and evaluated left to right, matching how they would be interpreted when spoken out loud. I parse the first number, followed by a list of operations and accompanying numbers using **many**. Each operation is stored as a partially applied constructor (e.g. **Add 5**, **Multiply 6**). This effectively builds a list of functions waiting for input. I then fold over this list from the left, feeding each result into the next operation. .

State Management (Memory)

To implement memory, I used the **StateT Memory IO** monad transformer. This allows me to combine stateful computations with IO, avoiding the need to manually pass state between functions.

The memory itself is a custom data type, storing either: **single values** (e.g. **last result**, **API key**) or **structured data** (e.g. **facts**, **portfolio**) using maps. This design allows efficient access while keeping different parts of the memory logically separated as individual record fields.

For the `"that"` keyword, the parser detects if its present and passes the remaining string to the responder. The responder then replaces all occurrences of `"that"` with the last stored result and re-parses the expression.

Extended Functionality

Permanent Memory

To extend the memory of the chatbot from being limited to a single session, I implemented permanent memory using JSON through the `aeson` library. Memory is saved to disk and loaded when the program is run. This required defining `ToJSON` and `FromJSON` instances for all of my custom data types. An `emptyMemory` value is used when no prior memory exists. The functions `saveMemory` and `loadMemory` handle all file interactions. Memory is saved after the State updates (e.g. storing facts, API key, portfolio changes), ensuring the state is preserved across sessions.

FinanceBro Bot

The main extension is a stock market system built using the Finnhub API. The user provides an API key, which is stored in memory and used for all requests. A general API function constructs requests, sends them, and decodes JSON responses into custom types. Since the API requires stock tickers rather than company names, all requests first match a company name to its ticker before fetching data.

An API key has been provided for you in `APIKEY.txt`.

Features include: **stock information** (e.g. price, change, high/low), **cost of shares**, **company information**, **analyst opinions** (converted into readable summaries), **news articles** (stored temporarily in memory). News results are stored in memory and indexed, allowing the user to request more detail about specific articles.

Portfolio Management

The chatbot can also store a user's stock portfolio. Users can inform the chatbot that they have bought or sold shares, with data stored in memory. When buying additional shares, a new average price is calculated. The chatbot supports: **viewing portfolio holdings**, **calculating total portfolio value**, **profit/loss analysis**, **identifying best and worst performing stocks**.

To test theses features see `Parser.hs` for expected strings.

Libraries Used (Beyond Standard)

- `aeson` – JSON encoding/decoding (API + Memory)
- `mtl` — monad transformers (`StateT`)
- `request` – API calls
- `directory` / `filepath` – file storage for memory
- `bytestring` - reading and writing of JSON data to files

Personal Reflection

Overall, I found this project very interesting. The biggest challenge was understanding monad transformers, particularly `StateT`, and integrating error handling (failing with meaningful messages) across both `StateT` and IO without excessive pattern matching. If I had more time, I would improve the parser to make it smarter and more lenient. Although I used Megaparsec combinators like `choice` and `optional`, it remains relatively fixed.